Exp Programming Language Specification — Version 0.1

---

1. Introduction

Exp is a dynamically typed, expression-oriented programming language with a clean C-style syntax, modern null-safety features, metadata attributes, and a simple object model. It is designed to be readable, predictable, and beginner-friendly while still expressive enough for real applications.

This document defines the syntax, semantics, operators, types, classes, functions, attributes, error handling, and known limitations of Exp v0.1.

---

2. Lexical Structure

2.1 Identifiers

Identifiers consist of letters, digits, and underscores, and must not begin with a digit.

2.2 Keywords

```
var const func class static constructor
if else while for foreach in
try catch finally throw
break continue
attribute is not bool number char function null id counter from to
new when
```

2.3 Comments

- **Single-line:** `// comment`
- **Block:** `/* comment */`

---

3. Types

Exp is dynamically typed. Values may be:

- number
- char
- boolean
- object

- function pointer
- null
- void

There is **no undefined**.

---

4. Variables

4.1 Declaration

```
var x = 10
const y = 20
```

4.2 Scope

Variables are block-scoped.

4.3 Mutability

- `var` → mutable
- `const` → immutable

---

5. Functions

5.1 Declaration Forms

Block form

```
func add(a, b)
{
    return a + b
}
```

Expression form

```
func add(a, b) => a + b
```

5.2 Return Rules

- Functions must explicitly return a value.
- No implicit “last expression” return.
- Functions may return nothing (void).

5.3 Nullability

Parameters cannot receive `null` unless marked nullable:

```
func greet(name?)
{
}
```

5.4 Parameter Restrictions

Parameters **cannot** have attributes.

---

6. Classes

6.1 Syntax

```
class Point(x, y)
{
    static id
    static const type

    constructor(a, b)
    {
        this.x = a
        this.y = b
    }
}
```

6.2 Properties

- Constructor parameters become instance properties.
- Static properties use:
    - `static name`
    - `static const name`
- Static properties **cannot** use `var`.

6.3 Static Constructors

Static properties default to `null`.
Initialization must occur in a static constructor:

```
static constructor()
{
    id = 0
    type = "point"
}
```

6.4 Inheritance

Exp v0.1 does **not** support inheritance.

---

7. Enums

Enums are syntactic sugar for classes with static const numeric properties.

7.1 Example

```
enum Color
{
    red,
    green,
    blue = 10,
    yellow
}
```

7.2 Expansion

```
class Color()
{
    static const red = 0
    static const green = 1
    static const blue = 10
    static const yellow = 2
}
```

---

8. Arrays

8.1 Literal Initialization

```
[a, b, c]
```

8.2 Constructor Initialization

```
new Array(len)
```

Creates an array of length `len` filled with `null`.

Arrays are zero-indexed and dynamic.

---

9. Control Flow

9.1 If / Else

```
if (cond)
{
}
else
{
}
```

9.2 While

```
while (cond)
{
}
```

9.3 For (C-style)

```
for (var i = 0; i < 10; i += 1)
{
}
```

9.4 Foreach

```
foreach item in list
{
}
```

With counter

```
foreach item in list : counter c
{
}
```

With label

```
foreach item in list : id loop1
{
    break loop1
}
```

9.5 Known Limitation

Single-statement bodies without braces are buggy.
Always use `{}`.

---

10. Operators

10.1 Arithmetic

```
+ - * / %
```

10.2 Comparison

```
== != < <= > >=
```

10.3 Logical

`&` (and)
`|` (or)

10.4 Assignment

```
= += -=
```

10.5 Null-Safety and Conditional Operators

Safe Navigation (`?.`)

```
a = b?.c
a?.b = 0
```

Null-Coalescing (`??`)

```
a = b ?? c
```

Ternary (`? :`)

```
a = b ? c : d
```

Operator Precedence

1. `?.`
2. `??`
3. `? :`

---

11. Strings and Characters

11.1 Strings

- Double quotes
- Escape sequences supported
- No multiline strings

11.2 Characters

- Single quotes

---

12. Error Handling

12.1 Try / Catch / Finally

```
try
```

```
{
}
catch e
{
}
finally
{
}
```

12.2 Throw

Only `system::Exception` objects may be thrown.

```
throw new system::Exception("error")
```

No inheritance → no custom exception types.

---

13. Attributes

Attributes provide metadata for declarations.

13.1 Declaring Attributes

Attributes have **no body**:

```
attribute Author(string name)
```

13.2 Applying Attributes

```
@Author("Ben")
class Product()
{
}
```

Attributes may annotate:

- classes
- properties
- functions
- constructors
- attributes

Not parameters.

---

14. Built-in Attributes

14.1 `Translator`

Marks the string representation method.

```
@Translator
func repr()
{
    return this.name
}
```

14.2 `Equalizer`

Marks the equality override.

```
@Equalizer
func equals(other?)
{
    return this.x == other.x & this.y == other.y
}
```

14.3 `AllowFor`

Defines where an attribute may be applied.

Signature

```
attribute AllowFor(bool classes, bool props, bool funcs, bool
constructors, bool attr)
```

Usage

`AllowFor` must appear **before** the attribute declaration it annotates:

```
@AllowFor(true, false, true, false, false)
attribute Version(number v)
```

---

15. Known Limitations (v0.1)

- Single-statement bodies without braces are buggy
- Static property inline initialization unsupported